



Table of Contents

[Preface](#)

[Part 1: Python for Offensive Security](#)

[1](#)

[Introducing Offensive Security and Python](#)

[Understanding the offensive security landscape](#)

[Defining offensive security](#)

[The origins and evolution of offensive security](#)

[Use cases and examples of offensive security](#)

[Industry relevance and best practices](#)

[The role of Python in offensive operations](#)

[Key cybersecurity tasks that are viable with Python](#)

[Python's edge in cybersecurity](#)

[The limitations of using Python](#)

[Ethical hacking and legal considerations](#)

The key protocols of ethical hacking

Ethical hacking's legal aspects

Exploring offensive security methodologies

Significance of offensive security

The offensive security lifecycle

Offensive security frameworks

Setting up a Python environment for offensive tasks

Setting up Python on Linux

Setting up Python on macOS

Setting up Python on Windows

Exploring Python modules for penetration testing

Essential Python libraries for penetration testing

Case study – Python in the real world

Scenario 1 – real-time web application security testing

Scenario 2 – network intrusion detection

Summary

2

Python for Security Professionals – Beyond the Basics

Utilizing essential security libraries

Harnessing advanced Python techniques for security

Compiling a Python library

Advanced Python features

Decorators

Generators

Summary

Activity

Part 2: Python in Offensive Web Security

3

An Introduction to Web Security with Python

Fundamentals of web security

Python tools for a web vulnerability assessment

Wapiti

MITMProxy

SQLMap

Exploring web attack surfaces with Python

HTTP header analysis

HTML analysis

JavaScript analysis

Specialized web technology fingerprinting libraries

Proactive web security measures with Python

Input validation and data sanitization

Secure authentication and authorization

Secure session management

Secure coding practices

Implementing security headers

Summary

4

Exploiting Web Vulnerabilities Using Python

Web application vulnerabilities – an overview

SQL injection

XSS

IDOR

[A case study concerning web application security](#)

[SQL injection attacks and Python exploitation](#)

[Features of SQLMap](#)

[How SQLMap works](#)

[Basic usage of SQLMap](#)

[Intercepting with MITMProxy](#)

[XSS exploitation with Python](#)

[Understanding how XSS works](#)

[Reflected XSS \(non-persistent\)](#)

[Stored XSS \(persistent\)](#)

[Python for data breaches and privacy exploitation](#)

[XPath](#)

[CSS Selectors](#)

[Summary](#)

[5](#)

[Cloud Espionage – Python for Cloud Offensive Security](#)

[Cloud security fundamentals](#)

[Shared Responsibility Model](#)

Cloud deployment models and security implications

Encryption, access controls, and IdM

Security measures offered by major cloud providers

Access control in cloud environments

Impact of malicious activities

Python-based cloud data extraction and analysis

Risks of hardcoded sensitive data and detecting hardcoded access keys

Enumerating EC2 instances using Python (boto3)

Exploiting misconfigurations in cloud environments

Types of misconfigurations

Identifying misconfigurations

Exploring Prowler's functionality

Enhancing security, Python in serverless, and infrastructure as code (IaC)

Introducing serverless computing

Introduction to IaC

Summary

Part 3: Python Automation for Advanced Security Tasks

6

Building Automated Security Pipelines with Python Using Third-Party Tools

The art of security automation – fundamentals and benefits

The benefits of cybersecurity automation

Functions of cybersecurity automation

Cybersecurity automation best practices

What is an API?

Designing end-to-end security pipelines with Python

Integrating third-party tools for enhanced functionality

Why automate ZAP with Python?

Setting up the ZAP automation environment

Automating ZAP with Python

CI/CD – what is it and why is it important for security automation?

Integrating Beagle Security into our security pipeline

Automating testing with Python

Ensuring reliability and resilience in automated workflows

Robust error-handling mechanisms

Implementing retry logic

Building idempotent operations

Automated testing and validation

Documentation and knowledge sharing

Security and access control

Implementing a logger for security pipelines

Summary

7

Creating Custom Security Automation Tools with Python

Designing and developing tailored security automation tools

Integrating external data sources and APIs for enhanced functionality

Extending tool capabilities with Python libraries and frameworks

pandas

scikit-learn

Summary

Part 4: Python Defense Strategies for Robust Security

8

Secure Coding Practices with Python

Understanding secure coding fundamentals

Principles of secure coding

Common security vulnerabilities

Input validation and sanitization with Python

Input validation

Input sanitization

Preventing code injection and execution attacks

Preventing SQL injection

Preventing command injection

Data encryption and Python security libraries

Symmetric encryption

Asymmetric encryption

Hashing

Secure deployment strategies for Python applications

Environment configuration

Dependency management

Secure server configuration

Logging and monitoring

Summary

9

Python-Based Threat Detection and Incident Response

Building effective threat detection mechanisms

Signature-based detection

Anomaly detection

Behavioral analysis

Threat intelligence integration

Real-time log analysis and anomaly detection with Python

Preprocessing

Real-time analysis with the ELK stack

Anomaly detection techniques

Visualizing anomalies

Automating incident response with Python Scripts

Leveraging Python for threat hunting and analysis

Data collection and aggregation

Data analysis techniques

Automating threat hunting tasks

Orchestrating comprehensive incident response using Python

Designing an incident response workflow

Integrating detection and response systems

Logging and reporting

Generating incident reports

Summary

Index

Other Books You May Enjoy